W02B · 기본적인 추천 방법 (2)

> 딥러닝응용I(추천시스템) · 동덕여자대학교 데이터사이언스전공 · 유원상 교수 · 2026-2 2026년 9월 10일 목요일 10:30

이번 수업에서 연결할 것

“평균을 계산했을 뿐인데 학습이라고 할 수 있을까?”라는 질문에서 시작한다. 지난 시간의 집단 추천을 수식과 작은 표로 다시 이해하고, `MeanRatingPredictor`의 학습과 예측을 실제 MovieLens 데이터에서 추적한다. 이어서 영화의 **내용 특징**으로 추천하고, 네 방법의 결과를 같은 조건에서 비교한다.

수업을 마치면 집단 평균의 의미와 대체 규칙, `fit`이 추정하는 모수, 가려 둔 정답으로 평가하는 이유를 설명할 수 있어야 한다. 또한 내용 유사도와 예측 평점을 구별하고, 순위 지표와 추천 목록을 함께 읽을 수 있어야 한다.

계산 예제를 읽는 방법

2026-09-10 계산 설명 보완: 각 절의 예제는 작은 표에서 직접 계산한 뒤 실제 실험으로 연결한다. 별도 표시가 없는 작은 표·영화 A/B/C·벡터 그림은 원리를 설명하기 위해 만든 가상 자료이며 MovieLens 측정값이 아니다. 계산 중에는 충분한 자릿수를 유지하고 최종 표시만 소수 넷째 자리로 반올림한다. TF–IDF의 로그는 자연로그, NDCG의 할인은 밑이 2인 로그다.

학습 순서는 **무엇을 세는가 → 어떤 수를 나누는가 → 결과의 단위는 무엇인가 → 추천/평가에서 어떻게 해석하는가**이다. 중간고사에 유사한 계산 문제가 출제될 수 있으므로 결과 숫자뿐 아니라 계산 근거를 설명하는 연습을 한다. 아래 예제는 실제 시험 문항의 공개나 출제 확정을 뜻하지 않는다.

1. 사용자 집단별 추천: 어떤 평점을 평균하는가?

사용자 u, 영화 i, 관측 평점 r_{ui}, 사용자의 집단 $g(u)$를 생각하자. 학습 관측 집합을 D_{train}이라고 쓴다. $g(u)$는 성별·직업처럼 미리 주어진 속성으로 정할 수 있다. 이번 방식은 군집화 알고리즘으로 집단 자체를 학습하는 방법은 아니다.

영화 i에 대해 집단 g가 남긴 **학습 평점만** 모은 집합을 다음과 같이 정의한다.

$$
\begin{aligned}
D_{gi} &= \{(u, i, r_{ui}) \in D_{\text{train}} : g(u) = g\}, \\
n_{gi} &= |D_{gi}|, \\
\mu_{gi} &= \frac{1}{n_{gi}} \sum_{(u,i,r_{ui}) \in D_{gi}} r_{ui} \quad (n_{gi} > 0).
\end{aligned}
$$

여기서 분모는 집단의 전체 인원수가 아니다. **그 집단에서 해당 영화를 평가한 관측 수**다. 다음은 원자료와 별개로 만든 작은 학습 표다.

사용자	집단	영화	학습 평점
P	동쪽	A	3
Q	동쪽	A	5

사용자	집단	영화	학습 평점
P	동쪽	B	2
R	서쪽	A	1
S	서쪽	A	5

동쪽×A는 $\frac{3+5}{2} = 4$, 서쪽×A는 $\frac{1+5}{2} = 3$, 동쪽×B는 $\frac{2}{1} = 2$다. A 전체 평균은 $\frac{14}{4} = 3.5$, 학습 전체 평균은 $\frac{16}{5} = 3.2$다. `groupby(["집단", "영화"])`는 이 표를 두 키가 같은 묶음으로 나누는 연산이다. 영화 하나만 키로 사용하면 집단 차이가 사라진다.

계산 예제 1 · 같은 표에서도 평점 수와 평균의 순서는 달라진다

먼저 두 기준의 차이를 별도 가상 표에서 확인한다. 영화 M의 학습 평점은 3, 3, 4이고 N의 학습 평점은 5 하나라고 하자. 두 영화 모두 아직 보지 않은 후보이다.

영화	학습 평점	평점 수 점수	평균 점수
M	3, 3, 4	3	3.3333
N	5	1	5.0000

$$s_{\text{count}}(M) = 3 > 1 = s_{\text{count}}(N), \qquad s_{\text{mean}}(M) = \frac{3+3+4}{3} = \frac{10}{3} < 5 = s_{\text{mean}}(N).$$

평점 수 기준은 M을 먼저, 평균 기준은 N을 먼저 추천한다. 평균 5라는 숫자만으로 신뢰도가 높다고 말할 수는 없다. N의 관측은 단 하나이다. 이번 기본형은 최소 영화 표본 수 필터를 적용하지 않으며, **점수의 단위는 각각 개수와 평점**이다.

표본이 없거나 너무 적다면

최소 집단 표본 수를 $m = 2$로 정하면, 동쪽×B의 평점 하나는 집단 평균으로 쓰지 않는다. 이번 예측기의 규칙은 다음과 같다. n_i는 영화 i의 학습 관측 수이며, μ_i와 μ는 각각 영화 평균과 학습 전체 평균이다.

$$\hat{r}_{ui} = \begin{cases} \mu_{g(u),i}, & n_{g(u),i} \geq m, \\ \mu_i, & n_{g(u),i} < m \text{ and } n_i > 0, \\ \mu, & n_i = 0. \end{cases}$$

동쪽 사용자의 A는 4, 서쪽 사용자의 B는 영화 평균 2, 학습에 없는 C는 전체 평균 3.2로 예측한다. 동쪽×B도 결과는 2지만 **집단 평균이 아니라 영화 평균을 사용했다**. 같은 숫자라도 근거가 다를 수 있으므로 `predict_details`의 `level`을 확인한다.

계산 예제 2 · 분모와 대체 경로를 함께 적는다

위의 동쪽·서쪽 학습 표로 돌아가자. 최소 집단 표본 수는 2이며 예측할 사용자는 학습 표와 별개인 새 동쪽/서쪽 사용자라고 하자. 해당 집단의 영화별 통계와 영화 전체 통계를 차례로 확인한다.

예측 요청	집단×영화 관측 수	집단 기준 사용 가능?	선택한 계산	예측과 근거
새 동쪽 사용자, A	2	가능	$\frac{3+5}{2}$	4, 집단 평균

예측 요청	집단×영화 관측 수	집단 기준 사용 가능?	선택한 계산	예측과 근거
새 서쪽 사용자, A	2	가능	$\frac{1+5}{2}$	3, 집단 평균
새 동쪽 사용자, B	1	불가	B의 전체 학습 평점 $\frac{2}{1}$	2, 영화 평균
새 서쪽 사용자, B	0	불가	B의 전체 학습 평점 $\frac{2}{1}$	2, 영화 평균
새 동쪽 사용자, C	0	불가, 영화 관측도 없음	$\frac{3+5+2+1+5}{5}$	3.2, 전체 평균

예측 결과가 2라고 해서 집단 평균을 사용했다고 단정하면 안 된다. 집단 평균에만 최소 표본 수 2를 적용하며, 이번 모델의 영화 평균 대체에는 같은 조건을 다시 적용하지 않는다. 집단에 속하지만 A를 평가하지 않은 사람이 더 있어도 A 평균의 분모에는 포함하지 않는다.

집단을 세분하면 취향 차이를 포착할 가능성과 표본 부족이 함께 커진다. 같은 집단이라고 모두 같은 취향은 아니다. 최소 표본 수를 높인다고 오차가 반드시 작아지는 것도 아니다.

평균도 지도학습인 이유

집단×영화마다 하나의 모수 θ_{gi}를 학습한다고 생각하자. 그 묶음의 모든 평점을 같은 값으로 예측하는 **범주별 상수 회귀 모형**이다.

$$L_{gi}(\theta) = \sum_{(u,i,r_{ui})\in D_{gi}} (r_{ui} - \theta)^2,$$

$$\frac{dL_{gi}}{d\theta} = 2n_{gi}\theta - 2\sum_{(u,i,r_{ui})\in D_{gi}} r_{ui},$$

$$\frac{dL_{gi}}{d\theta} = 0 \implies \hat{\theta}_{gi} = \frac{\sum_{(u,i,r_{ui})\in D_{gi}} r_{ui}}{n_{gi}},$$

$$\frac{d^2L_{gi}}{d\theta^2} = 2n_{gi} > 0.$$

평균은 제곱오차를 최소화하는 유일한 해다. 평점이라는 정답 레이블을 이용해 모수를 추정하므로 지도학습이다. 학습에 반드시 신경망이나 경사하강법이 필요한 것은 아니다. 데이터가 달라지면 추정한 평균도 바뀐다. 표본이 없는 묶음에서는 이 손실로 모수를 결정할 수 없으므로 위의 대체 규칙을 추가한다. 대체 규칙 자체는 이번 수업의 모델 설계다.

평균 계산도 정답으로 모수를 추정하는 학습이다

같은 집단 × 같은 영화

학습 평점: 3, 5

학습할 값: θ 하나

$L(\theta) = (3-\theta)^2 + (5-\theta)^2$

$\theta = 3 \Rightarrow L = 4$

→

fit: 손실을 최소화

$L'(\theta) = 4\theta - 16 = 0$

$\hat{\theta} = 4$

$L(4) = 1 + 1 = 2$

학습 속성에 4를 저장

→

predict: 저장한 값 조회

입력: 새 사용자, 영화 ID

같은 집단이면 예측 4

이 단계에는 정답 불필요

정답 2 확인 뒤 절대오차 2

원본 표와 다른 독자적 예제. θ가 평균이라는 닫힌 해를 사용하므로 반복 최적화가 필요 없다.

교재의 집단별 추천을 최소제곱 회귀로 해석한 설명과 위의 작은 표는 이해를 돕기 위해 추가한 것이다.

계산 예제 3 · 왜 학습 결과가 평균 4인가?

동쪽×A의 학습 정답은 3과 5이다. 두 관측에 같은 값으로 예측할 때 제곱오차 합을 직접 비교한다.

후보 예측값	첫 오차 제곱	두 번째 오차 제곱	제곱오차 합
3	0	4	4
4	1	1	2
5	4	0	4

$$L(\theta) = (3-\theta)^2 + (5-\theta)^2 = 2(\theta-4)^2 + 2.$$

제곱인 첫 항은 음수가 될 수 없으므로 4에서 손실이 최소 2가 된다. 이 모형은 각 관측의 정답을 모두 맞추는 것이 아니라 **이 묶음에 공통으로 쓸 상수 하나의 제곱오차를 최소화**한다. `fit`은 이 최적값 4를 계산해 저장한다. test 정답은 이 계산에 들어가지 않는다.

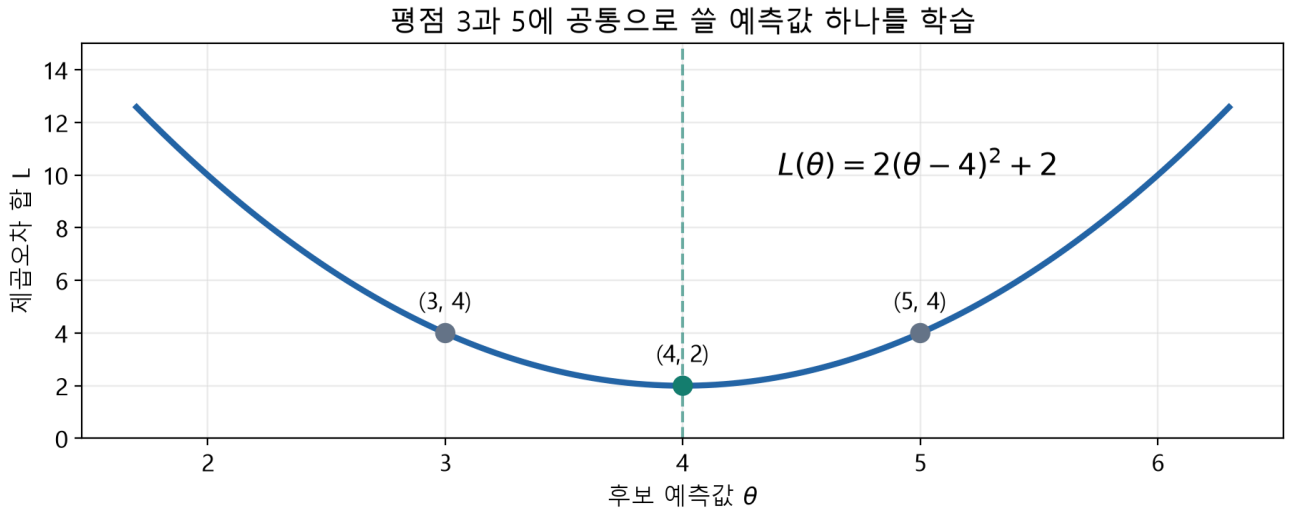


그림 읽기: 가로축은 후보 모수, 세로축은 학습 오차 합이다. 최저점이 학습되는 평균이며 관측 평점 자체의 위치와 구별한다.

2. MovieLens에서는 학습 데이터를 어떻게 준비하는가?

MovieLens 100K에는 사용자 943명, 영화 1,682편, 평점 100,000개가 있다. 학생 실행 시 공식 ZIP 또는 해시가 같은 고정 HTTPS 대체 경로에서 다운로드하며 원본 데이터를 수업 저장소에 재배포하지 않는다.

표	한 행의 의미	핵심 열	학습에서의 역할
`users`	사용자 한 명	`user_id`, `sex`, `occupation`	ID로 집단 속성을 연결
`movies`	영화 한 편	`movie_id`, `title`, 장르 19열	제목·내용 특징; `unknown`은 프로필에서 제외
`ratings`	사용자–영화의 관측 평점	`user_id`, `movie_id`, `rating`, `timestamp`	입력 쌍과 정답 레이블

평점 행 하나를 머신러닝의 관점에서 보면 X=(user_id, movie_id, g(user_id)), y=rating이다. ID 숫자의 크기를 연속형 특징으로 회귀하는 것은 아니다. ID는 학습 통계를 조회하는 범주 키다. 영화 장르는 예측 시 이미 알려진 카탈로그 정보로 가정한다.

처음 사용하는 준비 함수의 계약

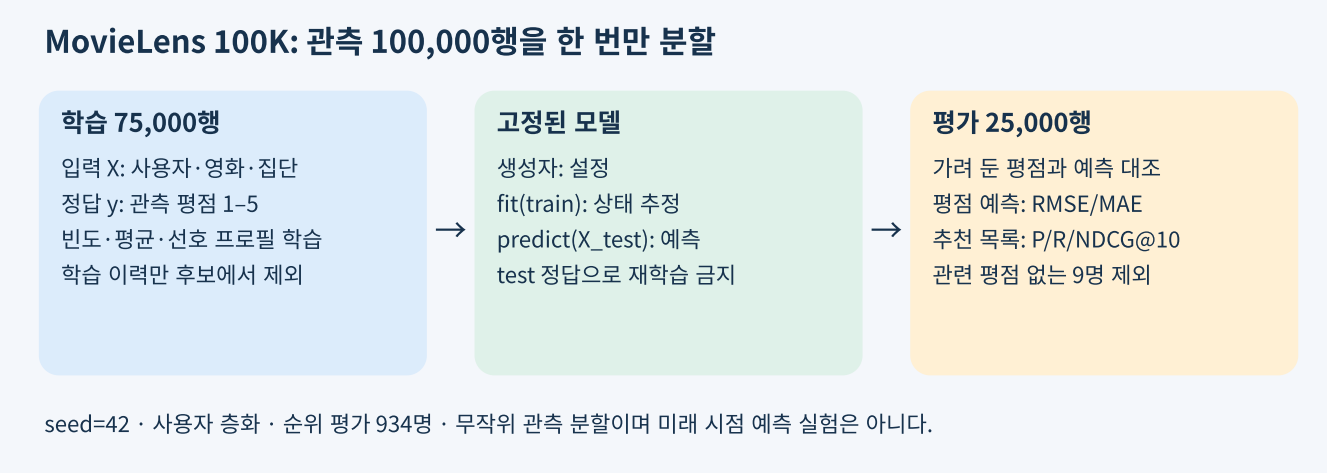
`prepare_movielens(cache_dir, local_dir=None, mode="auto", timeout=20)`는 데이터 준비 함수다. `cache_dir`는 캐시 폴더 경로, `local_dir`는 이미 가진 전체 데이터 폴더이며 일반 학생은 지정할 필요가 없다. API 기본값은 `auto`지만 **이번 실습은 `mode="real"`을 명시**하여 다운로드 실패를 합성 데이터로 숨기지 않는다. 반환 객체의 `.data`에는 `.users`, `.movies`, `.ratings` 표가 있고 `.mode`, `.description`은 실제 사용한 데이터 종류를 설명한다. 폴더 생성·다운로드는 이 함수의 부작용이다.

`split_ratings(ratings, test_size=0.25, seed=42)`는 관측 **행 위치**를 한 번 분할하고 `RatingSplit` 객체의 `.train`, `.test`, `.method`를 반환한다. 원본 표를 변경하지 않는다. MovieLens에서는 사용자를 층화하므로 같은 사용자의 관측이 학습·평가 양쪽에 들어가지만, 동일 사용자–영화 쌍은 겹치지 않는다. 사용자 전체를 가리는 새로운 사용자 평가와 다르다. 아주 작은 자료에서는 층화할 수 없어 `.method="random-small-data"`가 될 수 있다.

정의·인자별 설명은 준비와 분할 API 안내를 참조한다. notebook의 각 import 앞에서도 같은 입력·출력과 정의 위치를 확인할 수 있다.

```
from luna_recsys import prepare_movielens, split_ratings

prepared = prepare_movielens("data/local", mode="real")
data = prepared.data
split = split_ratings(data.ratings, test_size=0.25, seed=42)
print(len(split.train), len(split.test), split.method)
# 실제 결과: 75000 25000 user-stratified
```



이것은 무작위 관측 holdout이다. `timestamp` 순서대로 미래를 예측하는 실험은 아니다. 실제 서비스의 미래 추천을 주장하려면 시간 분할·노출 조건 등도 검토해야 한다.

3. MeanRatingPredictor를 생성·학습·예측·평가하기

`MeanRatingPredictor`는 **클래스**다. `MeanRatingPredictor(...)`를 호출하면 설정을 가진 객체가 만들어지고, `fit(...)`을 호출해야 평균이 학습된다. 함수 하나를 불러오는 것과 객체의 메소드를 호출하는 것을 구별하자.

코드	입력	하는 일	출력·상태
`MeanRatingPredictor("group", group_col="sex", min_group_ratings=2)`	기준·집단 열·최소 표본	설정 검사	아직 평균 없는 객체
`model.fit(split.train, data.users)`	학습 평점 표, 사용자 속성 표	정답으로 평균 추정	`self`; 아래 학습 속성 생성
`model.predict(pairs)`	`user_id`, `movie_id` 두 열	저장한 평균 조회	입력 순서의 예측 Series
`model.predict_details(pairs)`	같은 두 열	예측과 대체 수준 조회	`prediction`, `level` DataFrame

`mode`의 기본값은 `"movie"`, `group_col`은 `"sex"`, `min_group_ratings`는 1이다. `"global"`은 전체 평균, `"movie"`는 영화 평균, `"group"`은 집단×영화 평균을 사용한다. `group`에서는 사용자 ID가 유일하고 집단 값이 빠지지 않은 `users`가 필요하다. `predict`에 아직 학습하지 않은 객체를 사용하면 오류가 난다.

`fit` 후에는 다음 상태가 생긴다. 마지막 밑줄은 학습 후 속성이라는 명명 관례다.

속성	내용	표 연산과의 연결
`global_mean_`	숫자 하나	`train.rating.mean()`
`movie_means_`	영화 ID별 Series	`train.groupby("movie_id").rating.mean()`
`user_groups_`	사용자 ID→집단	사용자 표의 집단 열
`group_means_`	집단·영화 다중 인덱스 Series	속성을 연결한 뒤 두 키로 평균/개수 집계

```
from luna_recsys import MeanRatingPredictor

model = MeanRatingPredictor("group", group_col="sex", min_group_ratings=2)
model.fit(split.train, data.users)                  # y_train으로 모수 추정
pairs = split.test[["user_id", "movie_id"]]       # y_test를 제거한 입력
details = model.predict_details(pairs)            # 학습 상태를 그대로 사용
y_test = split.test["rating"].to_numpy()           # 이제 정답을 평가에만 사용
y_hat = details["prediction"].to_numpy()
errors = y_test - y_hat
mae = abs(errors).mean()
rmse = (errors ** 2).mean() ** 0.5
```

`fit`에는 정답을 주고, `predict`에는 정답을 주지 않는다. 평가자가 `y_test`를 별도로 보관했다가 예측과 비교한다. `to_numpy()`는 여기서 행 순서대로 숫자 배열을 얻는 연산이다. 표 인덱스가 섞인 상태에서 잘못 정렬하지 않도록 입력 순서를 보존한다.

작은 오차 예: 실제 평점이 `[5, 2, 4]`, 예측이 `[4, 3, 4]`라면 오차는 `[1, −1, 0]`이다. MAE는 $\frac{2}{3} \approx 0.667$, RMSE는 $\sqrt{\frac{2}{3}} \approx 0.816$이다. 큰 오차를 제곱하므로 RMSE가 더 크게 반응한다. 학습 평점을 다시 예측한 오차만 보면 이미 사용한 정답에 잘 맞는 정도를 측정하게 된다.

계산 예제 4 · fit 결과를 고정한 뒤 test에서 채점한다

위의 동쪽·서쪽 학습 표로 집단 모형을 학습했고 최소 집단 표본 수는 2라고 하자. 평가자가 보관한 새로운 세 관측은 다음과 같다. C는 카탈로그에 있지만 학습 평점은 없는 영화이다.

test 입력: 집단, 영화	가려 둔 실제 평점	train에서 찾은 예측	선택한 근거	오차: 실제−예측	절댓값	제곱
동쪽, A	5	4	집단 평균	1	1	1
서쪽, B	4	2	영화 평균	2	2	4
동쪽, C	3	3.2	전체 평균	−0.2	0.2	0.04

$$\mathrm{MAE} = \frac{|5-4| + |4-2| + |3-3.2|}{3} = \frac{3.2}{3} \approx 1.0667,$$

$$\mathrm{RMSE} = \sqrt{\frac{(5-4)^2 + (4-2)^2 + (3-3.2)^2}{3}} = \sqrt{\frac{5.04}{3}} \approx 1.2961.$$

분모 3은 평가한 평점 쌍의 수이며 학습 표의 5행이나 사용자 집단 크기가 아니다. 두 지표 모두 최종 단위는 평점이다. 두 번째 관측의 오차 2는 제곱하면 4가 되어 큰 오차의 영향이 커진다. 부호 있는 오차를 먼저 평균하면 양·음이 상쇄되므로 MAE가 되지 않는다.

누수를 숫자로 확인: 동쪽×A의 test 정답 5를 몰래 학습에 더하면 평균이 4에서 $\frac{13}{3}$으로 바뀌고, 그 test 행의 절대오차가 1에서 $\frac{2}{3}$으로 작아진다. 그러나 정답을 본 뒤 예측한 것이므로 이 개선을 일반화 성능으로 보고할 수 없다. test 표의 평점은 `predict` 이후 평가 단계에서만 꺼낸다.

실제 평균 예측기의 결과

다음은 동일 분할, 집단 `sex`, 최소 집단 표본 **1**일 때의 실제 실행값이다. 위 코드의 표본 2 예와 설정을 구별한다. `evaluate_means(split, users, group_col="sex", min_group_ratings=1)`는 세 모델을 각각 학습해 같은 25,000행에서 RMSE와 대체 사용 건수를 반환하는 편의 함수다.

평점 예측 모형	RMSE	집단 평균 사용	영화 평균 사용	전체 평균 사용
전체 평균	1.131210	0	0	25,000
영화 평균	1.030792	0	24,953	47
집단×영화 평균	1.041019	24,882	71	47

이번에는 집단을 나누지 않은 영화 평균이 더 낮은 RMSE를 보였다. 세분화가 무조건 성능 개선은 아니다. `level`이 집단인 행이 많다는 사실도 정확도 향상을 뜻하지 않는다. 이 RMSE 표는 아래 네 방법의 순위 평가표와 다른 질문에 답한다.

누수와 모델 설정

전체 평점으로 평균을 만든 뒤 train/test를 나누면 이미 정답 일부가 학습 통계에 들어갔다. `fit(data.ratings)`를 한 모델은 평가 정답을 본 것이다. scikit-learn의 누수 안내처럼 분할을 먼저 하고 학습에 쓰는 통계는 train에서만 계산해야 한다.

최소 표본 수·장르 처리·선호 하한을 test 점수를 보며 고르면 test도 모델 선택에 사용한 셈이다. 값을 비교해 선택할 때는 train 안에 validation을 추가하고 마지막 test는 고정한다. 이번 주 비교의 설정은 실행 전에 고정했다.

4. 내용 기반 필터링: 비슷한 내용을 찾는다

교재 2.5는 영화 줄거리를 특징으로 바꾸고, 좋아하는 영화와 내용이 비슷한 영화를 찾는 흐름을 설명한다. 다른 사용자의 평가 패턴을 직접 비교하는 협업 필터링과 입력 신호가 다르다. 내용 특징은 텍스트뿐 아니라 장르·태그·속성일 수도 있다. Google의 내용 기반 추천 설명에서도 사용자 행동과 아이템 특징을 같은 공간에서 연결한다.

교재의 텍스트 특징: TF-IDF

각 문서에서 자주 나오면서 모든 문서에 흔하지 않은 단어에 상대적으로 큰 가중치를 준다. scikit-learn 기본 `smooth_idf=True`에서는 다음 IDF를 사용하고, 기본 `norm="l2"`로 각 문서 벡터를 정규화한다.

$$
\begin{aligned}
\mathrm{idf}(t) &= \log \frac{1+N}{1+\mathrm{df}(t)} + 1, \\
\mathrm{tfidf}(t,d) &= \mathrm{tf}(t,d)\,\mathrm{idf}(t), \\
\cos(\mathbf{x},\mathbf{z}) &= \frac{\mathbf{x}^{\top}\mathbf{z}}{\|\mathbf{x}\|_2\|\mathbf{z}\|_2}.
\end{aligned}
$$

N은 문서 수, $\mathrm{df}(t)$는 단어 t를 포함한 문서 수, $\mathrm{tf}(t,d)$는 문서 d의 단어 t 빈도다. 코사인은 두 벡터가 영벡터가 아닐 때 정의한다.

`TfidfVectorizer()`의 `fit_transform(documents)`는 문자열 문서 목록에서 어휘·IDF를 추정하고 문서 수×어휘 수의 희소행렬을 반환한다. `get_feature_names_out()`은 열의 단어 이름을 반환한다. 이 `fit`은 평점 정답을 사용하지 않는 특징 추정이다. **같은 메소드 이름 `fit`이 모두 지도학습을 뜻하지 않는다.** 공식 API에서 기본값을 확인할 수 있다.

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = ["space robot adventure", "space adventure", "family comedy"]
vectorizer = TfidfVectorizer()
features = vectorizer.fit_transform(documents)
similarity_to_first = (features @ features[0].T).toarray().ravel()
```

위 문장은 직접 만든 예다. 첫 문서와 두 번째 문서는 단어를 공유하며, 세 번째는 공유하지 않아 내적이 0이다. 첫 문서 자신은 유사도가 1이므로 실제 추천에서는 ID를 기준으로 제외해야 한다. 교재의 모든 영화 쌍 유사도 행렬 대신 한 문서의 열만 계산하면 큰 행렬을 만들지 않아도 된다.

계산 예제 5 · 세 문장으로 TF-IDF를 끝까지 계산하기

바로 위 코드의 문장을 영화 A, B, C의 설명이라고 하자. **단어 순서는 모든 행에서 동일하게** adventure, comedy, family, robot, space로 둔다. 기본 TF는 문서 안의 실제 등장 횟수이다.

영화	문장	adventure	comedy	family	robot	space
A	space robot adventure	1	0	0	1	1
B	space adventure	1	0	0	0	1
C	family comedy	0	1	1	0	0
포함한 문서 수	문서별 존재 여부를 합함	2	1	1	1	2

① **IDF를 구한다.** 문서는 3개이다. adventure와 space는 두 문서에, 나머지는 한 문서에 나온다. 여기서 로그는 자연로그이다.

$$\mathrm{idf}(\text{space}) = \ln\frac{1+3}{1+2} + 1 = \ln\frac{4}{3} + 1 \approx 1.2877,$$
$$\mathrm{idf}(\text{robot}) = \ln\frac{1+3}{1+1} + 1 = \ln 2 + 1 \approx 1.6931.$$

robot은 더 적은 문서에 등장하므로 더 큰 가중치를 받는다. space가 A에 두 번 나오더라도, A와 B 두 문서에만 있다면 문서 빈도는 여전히 2이다. 그때 달라지는 것은 A 안의 TF이다. 기본 smoothing 식에서는 모든 문서에 등장한 단어도 IDF가 1이며 0이 아니다. TF-IDF 공식 계산 설명.

② **TF와 IDF를 곱한다.** 이 예제의 0/1 TF에서는 1인 칸을 해당 IDF로 바꾸면 된다. 아래는 정규화 전 벡터이다.

$$\mathbf{v}_A \approx (1.2877, 0, 0, 1.6931, 1.2877),$$
$$\mathbf{v}_B \approx (1.2877, 0, 0, 0, 1.2877),$$
$$\mathbf{v}_C \approx (0, 1.6931, 1.6931, 0, 0).$$

③ **L2 정규화한다.** 각 영화의 벡터를 자기 길이로 나누어 길이를 1로 맞춘다. A의 경우는 다음과 같다.

$$\|\mathbf{v}_A\|_2 = \sqrt{1.2877^2 + 1.6931^2 + 1.2877^2} \approx 2.4866, \qquad \mathbf{x}_A = \frac{\mathbf{v}_A}{\|\mathbf{v}_A\|_2}.$$

영화	adventure	comedy	family	robot	space
A	0.5179	0	0	0.6809	0.5179
B	0.7071	0	0	0	0.7071
C	0	0.7071	0.7071	0	0



그림 읽기: 열 하나가 같은 단어를 뜻한다. A와 B는 adventure와 space 열에서 겹치고 C는 겹치지 않는다. 각 패널의 색상 척도는 다르므로 패널 사이의 색 농도 자체를 수치로 비교하지 말고 칸에 적힌 값을 읽는다. 이것은 **5차원 성분 표**이며 2차원 공간으로 투영한 그림이 아니다.

④ **A와의 코사인 유사도를 구한다.** 두 벡터가 이미 L2 정규화되었으므로 내적만 계산하면 된다.

$$\begin{aligned}\cos(A, B) &= \mathbf{x}_A^\top \mathbf{x}_B \\ &\approx 0.5179 \times 0.7071 + 0.5179 \times 0.7071 \approx 0.7324, \\ \cos(A, C) &= 0, \qquad \cos(A, A) = 1.\end{aligned}$$

두 공통 단어 외에는 같은 위치에서 동시에 0이 아닌 성분이 없다. 따라서 코드의 `similarity_to_first`는 약 `[1.0000, 0.7324, 0.0000]`이다. 자신 A를 제외하면 B가 추천된다. **0.7324는 좋아할 확률 73.24%나 예측 평점이 아니다.** A와 B의 특징 방향이 얼마나 비슷한지를 나타낸다. 공유하는 두 단어의 TF가 같아도 A에는 robot 성분이 있어 두 벡터의 방향이 완전히 같지는 않다.

아래 코드를 앞의 코드 다음에 실행하면 계산 중간 상태를 확인할 수 있다. `idf_`는 학습한 단어별 IDF이고, `toarray()`는 이 작은 3×5 희소행렬을 보통의 배열로 바꾼다. 대규모 영화 행렬을 무조건 밀집 배열로 바꾸지는 않는다. `round(4)`는 표시용 반올림이며 추천 점수를 정렬하기 전에 적용하지 않는다.

```
print(vectorizer.get_feature_names_out())  # 열의 단어 순서
print(vectorizer.idf_.round(4))            # [1.2877, 1.6931, 1.6931, 1.6931, 1.2877]
print(features.toarray().round(4))         # 위의 정규화된 세 행
print(similarity_to_first.round(4))        # [1.0000, 0.7324, 0.0000]
```

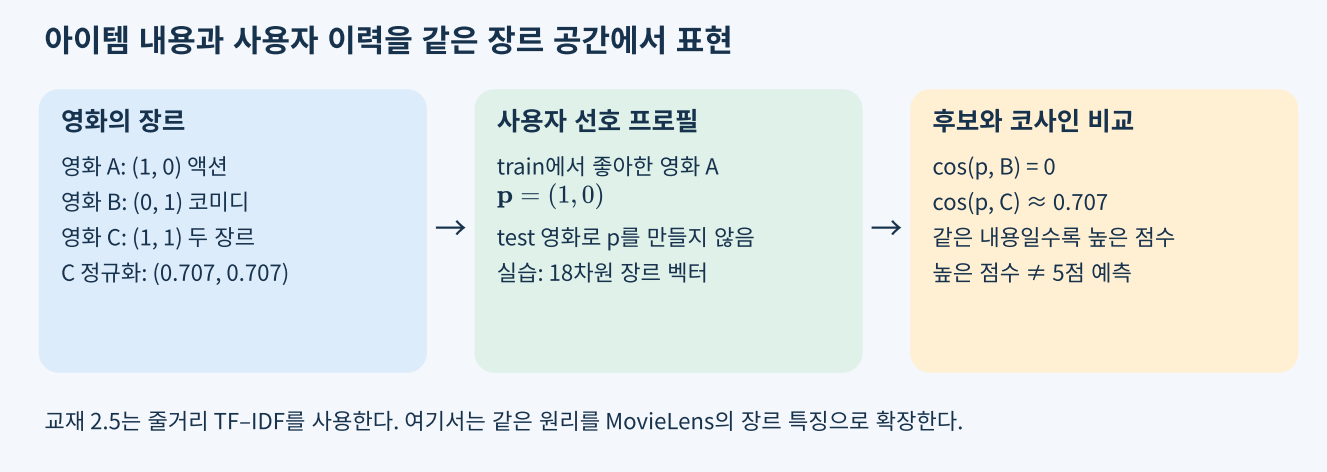
`fit`은 세 문장에서 어휘와 IDF를 추정하고 `transform`은 그 기준으로 문장을 벡터로 만든다. `fit_transform`은 두 과정을 이어서 수행한다. 여기에는 사용자 평점 정답이 없다. 이후 새로운 문장이 오면 기존 어휘/IDF를 유지하는 `transform`을 사용하며, 모르는 단어는 기본적으로 새 열을 만들지 않는다.

동일 MovieLens 비교에서는 장르를 사용한다

MovieLens 100K에는 줄거리 열이 없다. 교재의 `movies_metadata.csv`는 별도 데이터이므로 여기로 바꿔 계산한 점수를 같은 비교표에 넣지 않는다. 이번 실험은 **19개 장르 중 unknown을 제외한 18개 multi-hot 특징**을 사용한다. 한 영화에 장르가 여러 개면 1이 여러 개이므로 one-hot과 다르다.

$$
\begin{aligned}
\mathbf{x}_i &\in \{0,1\}^{18}, \\
\mathbf{v}_i &= \begin{cases} \mathbf{x}_i / \|\mathbf{x}_i\|_2, & \|\mathbf{x}_i\|_2 > 0, \\ \mathbf{0}, & \|\mathbf{x}_i\|_2 = 0, \end{cases} \\
H_u &= \{ i : (u, i, r_{ui}) \in D_{\text{train}},\ r_{ui} \geq 4 \}, \\
\mathbf{q}_u &= \frac{1}{|H_u|} \sum_{i \in H_u} \mathbf{v}_i \quad (|H_u| > 0), \\
\mathbf{p}_u &= \frac{\mathbf{q}_u}{\|\mathbf{q}_u\|_2} \quad (\|\mathbf{q}_u\|_2 > 0), \\
s(u, j) &= \mathbf{p}_u^{\top} \mathbf{v}_j .
\end{aligned}
$$

먼저 영화를 정규화하여 장르 수가 많은 영화 하나가 단순히 더 큰 벡터를 갖지 않도록 한다. 좋아한 영화 벡터를 평균하고, 방향을 비교하기 위해 사용자 프로필도 정규화한다. 낮게 평가한 영화를 직접 빼는 모델은 아니다. 4점 미만은 이 프로필에서 사용하지 않는다.



프로필이 비었거나 영벡터이면 해당 사용자의 내용 추천 전체를 영화 평균으로 대체한다. 정상 프로필에서 장르가 없는 후보는 유사도 0으로 둔다. 실제 카탈로그에는 알려진 장르가 없는 영화 2편, 전체 학습 사용자 중 유효 프로필 없는 사용자 1명이 있었다. 순위 평가에 포함된 934명에서는 빈 프로필 대체가 발생하지 않았다.

계산 예제 6 · 영화 벡터에서 사용자 프로필과 추천 순위까지

18차원 실험을 이해하기 위해 **액션·코미디만 있는 독립적인 2차원 가상 카탈로그**를 생각하자. 좌표 순서는 항상 (액션, 코미디)이다. 사용자가 train에서 4점 이상 준 서로 다른 세 영화는 액션 전용 두 편과 코미디 전용 한 편이다. 모두 단일 장르라 정규화해도 (1,0), (1,0), (0,1)이다. 실제 영화 제목이나 MovieLens 관측을 사용한 예가 아니다.

① 선호 영화 벡터의 평균을 만든다.

$$\mathbf{q}_u = \frac{(1,0)+(1,0)+(0,1)}{3} = \left(\frac{2}{3}, \frac{1}{3}\right).$$

② 사용자 프로필을 정규화한다.

$$\|\mathbf{q}_u\|_2 = \frac{\sqrt{5}}{3}, \qquad \mathbf{p}_u = \left(\frac{2}{\sqrt{5}}, \frac{1}{\sqrt{5}}\right) \approx (0.8944, 0.4472).$$

③ 아직 보지 않은 후보의 장르도 정규화하고 내적을 구한다. 이미 좋아한 세 영화는 후보에서 제외하며, 아래는 그 영화들과 다른 ID의 후보이다.

후보	원래 장르 벡터	정규화한 벡터	사용자와 코사인 점수	순위
D: 액션 전용	$(1,0)$	$(1,0)$	0.8944	2
E: 액션·코미디	$(1,1)$	$(0.7071, 0.7071)$	0.9487	1
F: 코미디 전용	$(0,1)$	$(0,1)$	0.4472	3

$$s(u,E) = \frac{2}{\sqrt{5}}\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{5}}\frac{1}{\sqrt{2}} = \frac{3}{\sqrt{10}} \approx 0.9487.$$

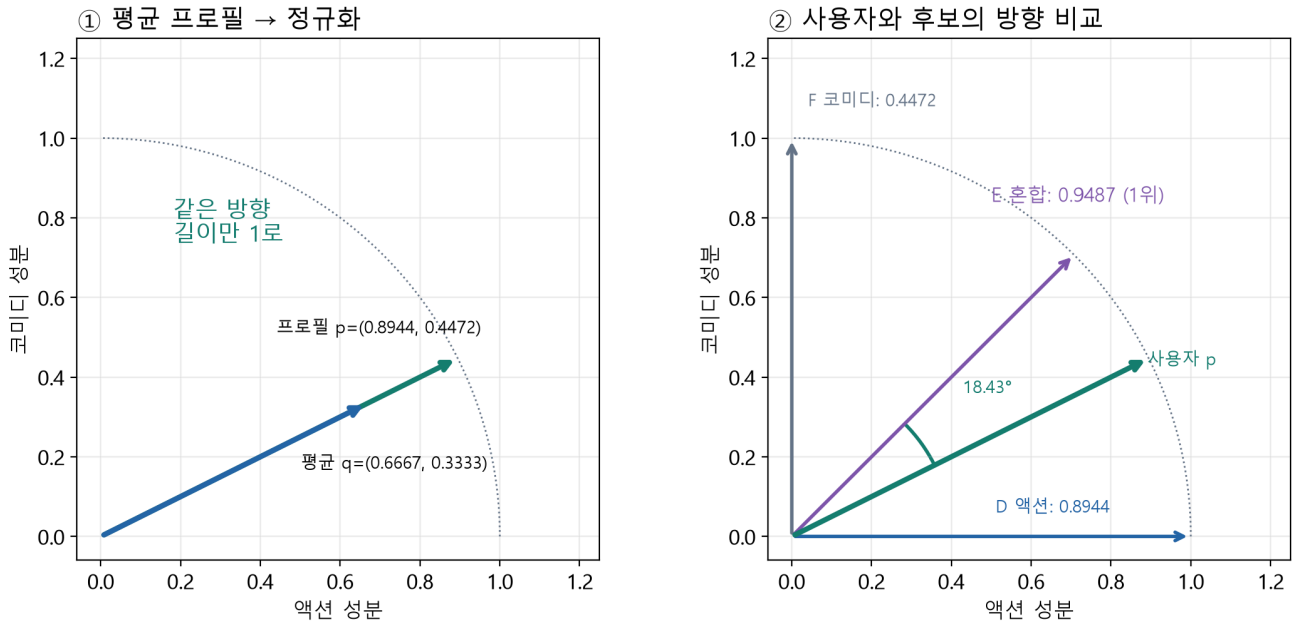


그림 읽기: 왼쪽에서 정규화는 프로필 화살표의 **길이를 바꾸지만 방향은 유지**한다. 오른쪽에서 사용자 방향은 액션 축으로부터 약 26.57도, 혼합 장르 E는 45도이므로 둘의 차이는 약 18.43도이다. D와의 차이 26.57도보다 작아 E가 더 비슷하다. 액션을 더 많이 좋아했더라도 코미디 선호도 함께 반영되므로 혼합 장르가 1위일 수 있다. 이 그림은 바로 이 2차원 예제의 실제 좌표이며 앞의 5차원 TF-IDF 벡터를 그린 것이 아니다.

후보 E를 정규화하지 않고 (1,1)과 바로 내적하면 약 1.3416이 된다. 이것은 코사인 점수가 아니며, 장르가 많아 생긴 길이의 영향이 섞인다. 두 비영벡터의 정규화된 내적은 이 비음수 특징 예제에서 0부터 1 사이이다. 영벡터는 길이로 나눌 수 없으므로 구현의 별도 대체 규칙을 사용한다.

5. 네 방법을 공정하게 비교하는 실험

`FourMethodRecommender().fit(train, users, movies)`는 같은 학습 자료로 네 기준을 준비한다. `recommend(user_id, method, k=10)`은 이미 학습에서 평가한 영화를 제외하고 `movie_id`, `score`, `basis`, `title`, `genres` 열의 표를 반환한다. `method`는 아래 네 문자열 중 하나다. `score_catalog(user_id, method)`는 후보 제거 전 모든 영화의 점수를 반환해 내부 동작을 점검할 때 사용한다.

method	점수를 학습하는 방법	점수 단위	개인별로 달라지는 부분
`count`	영화별 train 관측 수	평점 개수	이미 본 영화 제외
`mean`	영화별 train 평점 평균	예측 평점	이미 본 영화 제외
`group`	사용자 집단×영화의 train 평균	예측 평점	집단 및 본 영화 제외
`content`	train 선호와 장르 프로필	코사인 유사도	사용자 선호 및 본 영화 제외

이번 비교는 최소 관측 수로 후보를 추가 제거하지 않은 **기본형**이다. 영화 평균은 표본 하나의 5점도 5로 추정한다. 지난 수업 앱의 최소 평점 수 필터를 적용한 결과와 섞지 않는다. 집단 평균 최소 표본은 1이다. 학습 관측이 없는 영화는 count에서 0, 평균 계열에서 전체 평균으로 대체한다.

공통 평가 조건

1. `split_ratings`를 한 번만 실행한다: train 75,000행, test 25,000행, seed 42.
2. 전체 1,682편 카탈로그에서 사용자별 **train 이력만** 제외한다. test 이력을 제외하면 추천해야 할 정답 후보가 사라진다.
3. 점수 내림차순, 동점은 영화 ID 오름차순으로 정한다. 표시용 반올림 전에 정렬한다.
4. test에서 4점 이상 준 영화를 관련 아이템으로 정한다. 관련 영화가 없는 9명은 모든 방법에서 똑같이 제외하여 934명을 평가한다.
5. 각 사용자에서 K=10으로 측정한 뒤 사용자별 단순평균을 낸다. 많은 평점을 남긴 사용자에게 평균에서 더 큰 가중치를 주지 않는다.

순위 지표를 작은 예로 이해하기

관련 영화가 `{A,C}`, 추천이 `[A,B,C]`이고 K=3이면 적중은 두 개다. Precision은 $\frac{2}{3}$, Recall은 $\frac{2}{2} = 1$이다. NDCG는 같은 적중도 앞에 놓을수록 높게 평가한다. 이진 관련성의 $\mathrm{DCG@}K = \sum_{k=1}^{K} \frac{\mathrm{hit}(k)}{\log_2(k+1)}$를 관련 영화가 맨 위에 있는 이상적 순위의 IDCG로 나눈다.

추천 위치를 보며 순위 품질을 계산하는 작은 예

관련 영화 {A, C}

추천: A, B, C (K=3)

각 위치의 적중: 1, 0, 1

$\text{Precision} = \frac{2}{3}$

$\text{Recall} = \frac{2}{2} = 1$

→

위치별 할인

$k=1: \quad \frac{1}{\log_2 2} = 1$

$k=2: \quad \frac{0}{\log_2 3} = 0$

$k=3: \quad \frac{1}{\log_2 4} = 0.5$

$\text{DCG} = 1.5$

→

이상적 순위로 나누기

관련 2편이 1·2위라면

$\text{IDCG} = 1 + \frac{1}{\log_2 3}$

$\text{NDCG} \approx \frac{1.5}{1.631}$

$\text{NDCG} \approx 0.920$

실제 실험에서는 K=10. test에서 미관측인 후보는 관련성 미확인이며 싫어하는 영화라고 단정하지 않는다.

`ranking_metrics(recommended, relevant, k=10)`는 한 사람의 지표 사전을 반환한다. 빈 관련 집합은 오류로 처리하므로 제외 기준이 숨겨지지 않는다. `evaluate_rankings(model, test, k=10, relevance_threshold=4)`는 학습된 객체를 재학습하지 않고 **요약표, 사용자별 결과표, 평가 인원·조건 사전**을 반환한다. 자세한 수식은 NDCG 공식 API 설명과 비교할 수 있다. 우리 구현은 동점 정렬을 먼저 확정한 이진 관련성 계산이다.

계산 예제 7 · 적중 개수와 적중 위치는 다른 정보를 준다

관련 영화가 A와 C이고 추천 목록이 A, B, C이며 K=3인 위의 예를 끝까지 계산한다. 관련 여부는 test 평점으로 평가자가 정하며 추천기는 이 정답을 보지 않는다.

순위	추천 영화	적중 여부	할인 분모	DCG에 더할 값
1	A	1	$\log_2(2) = 1$	1.0000
2	B	0	$\log_2(3) \approx 1.5850$	0.0000
3	C	1	$\log_2(4) = 2$	0.5000

$$\text{Precision@3} = \frac{2}{3} \approx 0.6667, \qquad \text{Recall@3} = \frac{2}{2} = 1.$$

$$\text{DCG@3} = 1 + 0 + \frac{1}{2} = 1.5, \qquad \text{IDCG@3} = 1 + \frac{1}{\log_2 3} \approx 1.6309,$$

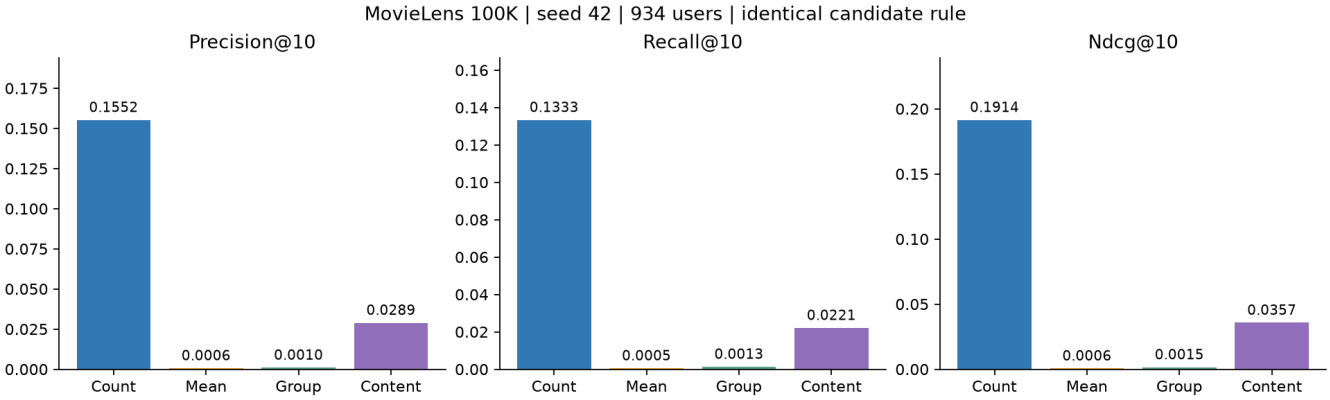
$$\text{NDCG@3} = \frac{1.5}{1.6309} \approx 0.9197.$$

이상적인 순서는 관련 영화 A와 C를 1·2위에 둔다. 두 영화 중 어느 것을 먼저 두어도 이진 관련성에서는 IDCG가 같다. 추천을 A, C, B로 바꾸면 적중 수가 같아 Precision과 Recall은 그대로지만 NDCG는 1이 된다. IDCG는 K 자체가 아니며, 관련 영화가 두 개뿐이면 이상적 적중도 두 개까지만 둔다.

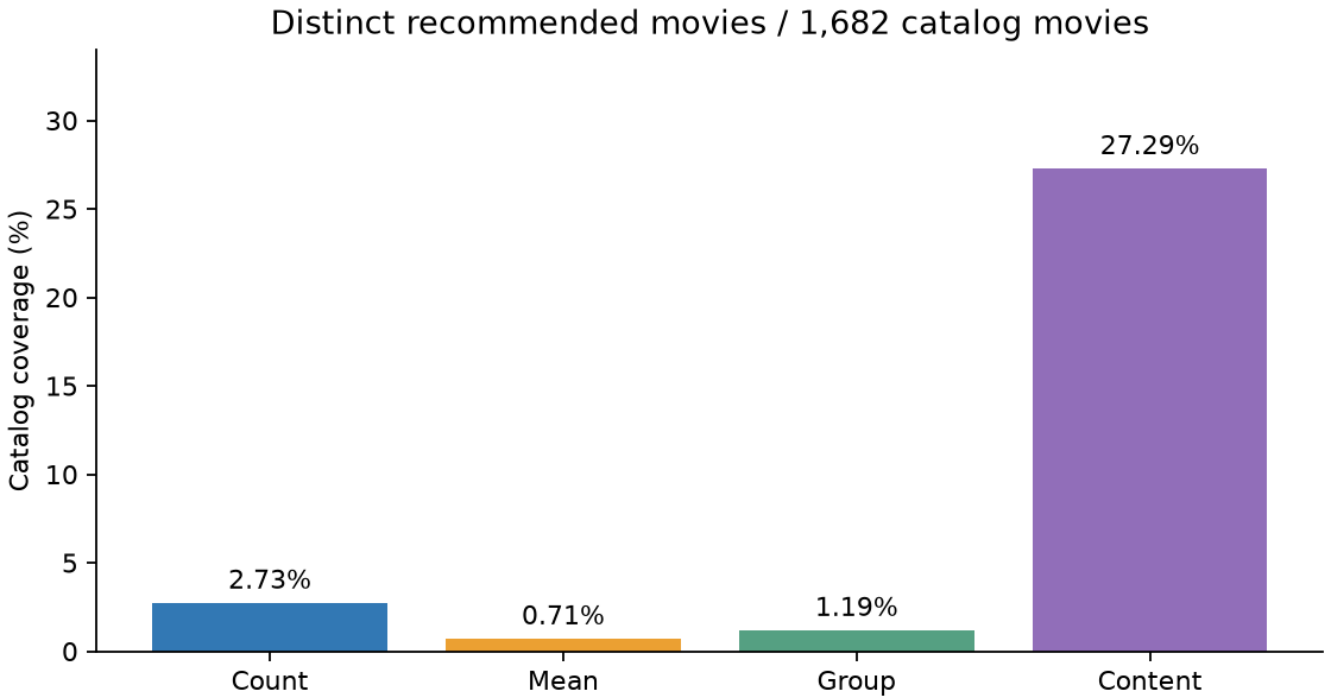
전체 지표는 사용자별 값을 단순평균한다. 예를 들어 같은 K=3에서 U의 Precision이 $\frac{2}{3}$, Recall이 $\frac{2}{2}$이고 V의 Precision이 $\frac{1}{3}$, Recall이 $\frac{1}{4}$이면 평균 Precision은 $\frac{1}{2}$, 평균 Recall은 $\frac{5}{8} = 0.625$이다. 적중을 모두 합친 3을 관련 영화 총수 6으로 나눈 0.5는 **사용자별 평균 Recall과 다른 집계 방식**이다.

실제 정량 결과: 2026-09-09 실행

방법	Precision@10	Recall@10	NDCG@10	카탈로그 커버리지
평점 수	0.155246	0.133349	0.191447	2.73%
영화 평균	0.000642	0.000505	0.000602	0.71%
집단×영화 평균	0.000964	0.001319	0.001524	1.19%
내용 기반	0.028908	0.022107	0.035705	27.29%



이 조건에서는 평점 수가 관측된 관련 영화의 회수에 가장 유리했다. 반면 표본 수 제약 없는 평균은 드물게 평가된 5점 영화가 상위를 차지해 test의 관측 관련 영화와 잘 겹치지 않았다. “평균 기반 추천은 항상 나쁘다”는 결론이 아니라 **이번 기본형·분할·관련성 관측 조건의 결과**다. 최소 표본 수나 수축 평균을 도입하는 후속 연구의 출발점이다.



내용 기반은 순위 지표가 평점 수보다 낮지만 전체 사용자에게 추천된 영화의 범위가 더 넓었다. 커버리지는 **전체 카탈로그에서 얼마나 넓게 추천했는가**이며 한 사용자의 목록 내 다양성이나 만족도 자체는 아니다. 집단 방식은 상위 추천 항목의 약 5.85%에서 영화/전체 평균으로 대체했다. 내용 방식의 상위 목록에 장르 영벡터 영화가 들어간 비율은 0이었다.

계산 예제 8 · Catalog coverage는 추천 영화의 합집합이다

정의: 평가 대상 사용자들에게 한 번이라도 추천된 서로 다른 영화가 전체 카탈로그에서 차지하는 비율이다. 사용자 한 명의 목록 길이를 세거나, 같은 영화가 추천된 횟수를 모두 더하는 지표가 아니다.

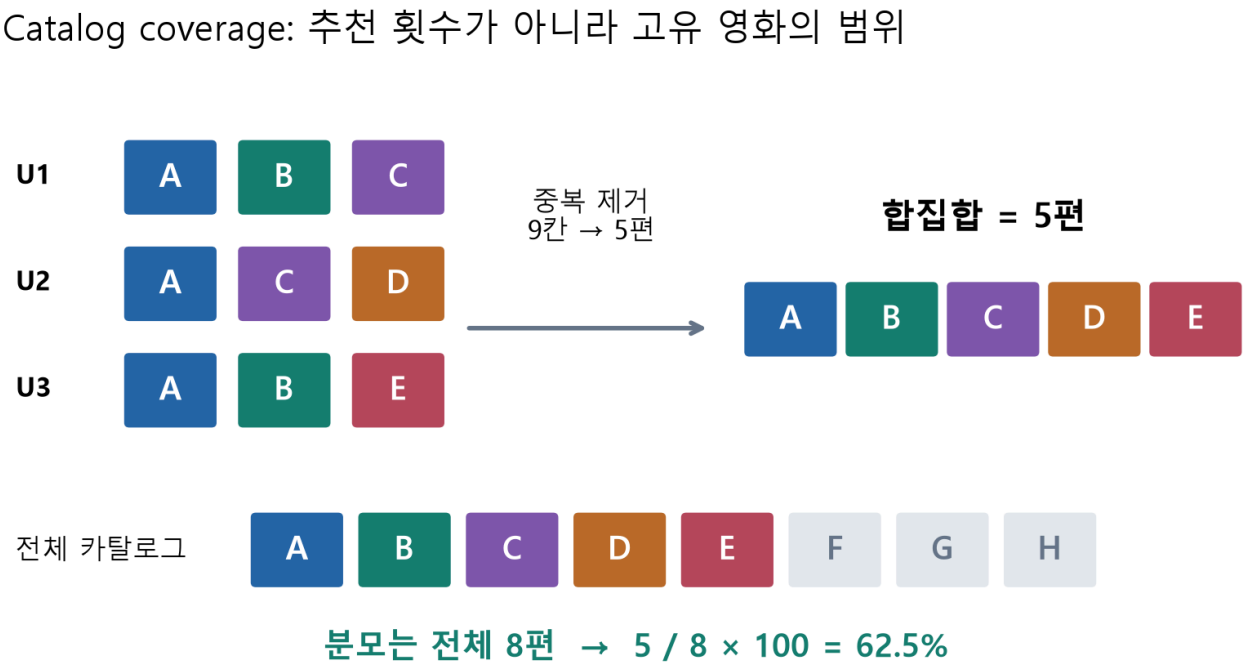
$$\mathrm{CatalogCoverage@}K = \frac{\left|\bigcup_{u \in U_{\mathrm{eval}}} R_K(u)\right|}{|I|} \times 100\%.$$

여기서 U_{eval}은 평가 대상 사용자 집합, $R_K(u)$는 사용자 u의 상위 K개 추천 영화 집합, I는 공통 전체 카탈로그이다. 합집합 기호는 목록을 합치면서 중복을 제거한다는 뜻이고, 세로 막대는 집합의 원소 수를 뜻한다. 표와 그림은 백분율로 표시하지만 `evaluate_rankings`가 반환하는 `catalog_coverage`는 0–1 비율이다.

전체 카탈로그가 A부터 H까지 8편이고 사용자 세 명에게 K=3으로 추천했다고 하자.

사용자	상위 3개 추천
U1	A, B, C
U2	A, C, D
U3	A, B, E

$$\{A,B,C\} \cup \{A,C,D\} \cup \{A,B,E\} = \{A,B,C,D,E\}, \qquad \mathrm{Coverage@3} = \frac{5}{8} \times 100\% = 62.5\%.$$



총 추천 칸은 9개지만 고유 영화는 5편이다. A는 세 명에게 추천되어도 한 편으로만 센다. 세 명 모두 A, B, C를 받았다면 커버리지는 $\frac{3}{8} \times 100\% = 37.5\%$이다. 두 경우 모두 개인 목록 길이는 3이지만 전체 노출 범위는 다르다. 실제 좋아했는지 여부는 커버리지 계산에 사용하지 않으므로 높은 값만으로 정확도나 만족도가 높다고 판단할 수 없다.

MovieLens 그림을 숫자로 읽기: 여기서는 평가 대상 934명, K=10, 전체 영화 1,682편이다. 이전 실행 결과의 비율을 고유 영화 수로 풀면 다음과 같다.

방법	전체 사용자 추천의 고유 영화 수	전체 카탈로그로 나눈 비율	백분율
평점 수	46	$\frac{46}{1682}$	2.73%
영화 평균	12	$\frac{12}{1682}$	0.71%
집단×영화 평균	20	$\frac{20}{1682}$	1.19%
내용 기반	459	$\frac{459}{1682}$	27.29%

분모는 934명이나 추천 칸 9,340개가 아니다. 예를 들어 내용 기반은 영화 459편을 최소 한 번 추천했다는 뜻이며, 추천 중 27.29%가 정답이라는 뜻이 아니다. 같은 방법도 평가 사용자 수나 K를 늘리면 커버리지가 달라질 수 있으므로 비교 시 **평가 사용자·K·카탈로그·후보 제외 조건**을 맞춘다. 인기 점수는 사용자에게 공통이어도 이미 본 영화가 서로 달라 제거되는 후보가 달라지므로, K=10인 평점 수 방식의 고유 추천 영화가 10편을 넘을 수 있다.

정성 비교: 목록을 직접 읽는다

notebook에서는 실험 전에 고른 두 익명 사용자 사례의 상위 목록을 나란히 출력한다. 개인 속성·개별 원본 평점은 공개 출력하지 않는다. **사례 A**에서 평점 수 방식의 선두는 *Star Wars (1977)*, 영화 평균 방식은 표본이 매우 적은 5점 영화 *Great Day in Harlem, A (1994)*였다. 제목뿐 아니라 `score`의 단위와 학습 표본 수를 함께 읽어야 한다.

다음 네 질문으로 두 사례를 비교한다. 실습 정답·해설은 교수자판에 별도로 제공한다.

- 같은 인기 점수인데 두 사용자 목록이 다르면, 학습에서 이미 본 영화 제외로 설명할 수 있는가?
- 평균 5점의 학습 표본 수는 충분한가? 관련 test 영화가 없다는 것이 낮은 품질의 증거인가?
- 내용 기반 상위 영화의 장르 조합은 사용자 프로필의 큰 성분과 연결되는가?
- 서로 다른 내용의 영화를 발견하게 하는가, 비슷한 장르만 반복하는가? 커버리지와 목록 내 다양성을 혼동하지 않았는가?

평가의 한계: MovieLens의 미관측 영화는 싫어한 영화가 아니다. 사용자는 모든 영화를 무작위로 평가하지 않았으며 노출·선택 편향이 남는다. 모든 미관측 후보를 관련성 0으로 채점한 이번 오프라인 지표가 실제 만족도의 참값은 아니다. 무작위 분할 하나의 결과에는 불확실성도 있다. 추가 분할·시간 분할·사용자 집단별 분석은 별도 실험으로 다룬다.

6. 결과를 웹 앱으로 연결하기

이번 앱은 지난 수업의 데이터 준비→추천 함수→화면 callback 구조를 이어간다. 학습은 한 번 수행하고, 사용자와 방법을 바꿀 때 저장된 모델에서 추천한다. `comparison_table(model, user_id, k=10)`은 네 목록을 하나의 표로 합치는 callback이며 `build_comparison_app(model)`은 Gradio 앱 객체를 반환한다. 앱 생성과 서버 실행은 분리된다.

실습에서는 callback의 k를 바꾸고 표의 순위·점수·근거를 확인한 뒤 버튼에 연결한다. `launch(share=True)`는 Colab에서 실행 중인 임시 서버로 연결되며 영구 배포 주소가 아니다. 같은 방법의 숫자는 비교할 수 있지만 평점 개수와 코사인 크기를 서로 비교하지 않는다.

계산 연습 · 숫자를 바꾸어 다시 풀기

다음은 복습용 새 문제이다. 바로 위의 완성 예제를 보지 않고 먼저 풀이를 적는다. **사용한 식, 분모/정규화 기준, 대체 경로, 결과 해석**을 함께 쓰고 최종값만 소수 넷째 자리로 반올림한다. TF-IDF 계산에는 자연로그, NDCG에는 밑 2 로그를 사용한다.

1. **집단 평균과 대체:** 학습 표가 (동쪽,A,2), (동쪽,A,4), (서쪽,A,5), (서쪽,B,1)이고 최소 집단 표본 수가 2이다. 새로운 동쪽 사용자의 A, B, 학습에 없는 C에 대한 예측과 각각의 근거를 구하라. 그 세 test 정답이 순서대로 4, 2, 5라면 MAE와 RMSE는 얼마인가?
2. **TF와 문서 빈도:** 세 문서가 `space space robot`, `space robot`, `family comedy`이다. space와 robot의 문서 빈도와 IDF를 구하라. 첫 문서의 정규화 전 TF-IDF, 정규화한 벡터, 두 번째 문서와의 코사인 유사도를 구하라. space의 총 등장 횟수를 문서 빈도로 사용하지 않는다.
3. **프로필과 벡터 비교:** 학습에서 좋아한 세 영화의 정규화된 장르 벡터가 (1,0), (0,1), (0,1)이다. 사용자 프로필을 구하고, 아직 보지 않은 후보의 원래 벡터 (1,0), (1,1), (0,1)에 대한 코사인 점수와 추천 순서를 구하라.
4. **순위와 커버리지:** 전체 카탈로그는 A부터 H까지 8편이고 K=3이다. U1의 관련 집합은 {A,D}, 추천은 [B,A,D]이다. U1의 Precision, Recall, DCG, IDCG, NDCG를 구하라. U2의 추천이 [A,C,F]일 때 두 사용자의 카탈로그 커버리지를 구하라. 커버리지를 구하는 데 U2의 관련 집합이 필요한지도 설명하라.

자가 점검: 평균의 분모는 해당 학습 관측 수, 평가 오차의 분모는 test 관측 수, Recall의 분모는 해당 사용자의 관련 영화 수, 커버리지의 분모는 전체 카탈로그 크기이다. 서로 바꾸어 쓰지 않았는지 확인한다. 계산기를 사용하더라도 중간 과정이 있어야 결과를 검토할 수 있다.

점검 퀴즈

1. 집단×영화 평균의 분모는 집단 전체 인원수인가, 어떤 관측 수인가?
2. 경사하강법 없이 평균을 계산하는 것도 지도학습이라고 할 수 있는 이유는 무엇인가?
3. MeanRatingPredictor 객체를 만든 직후와 fit 직후의 차이는 무엇인가?
4. predict에 test의 실제 평점이 필요하지 않은 이유는 무엇인가?
5. MovieLens 100K에서 교재의 줄거리 대신 장르를 사용한 이유는 무엇인가?
6. 내용 프로필을 test에서 좋아한 영화까지 포함하여 만들면 어떤 문제가 생기는가?
7. 평점 수와 코사인 점수를 RMSE에 바로 넣지 않고 순위 지표로 비교한 이유는 무엇인가?
8. 내용 기반의 카탈로그 커버리지가 높다는 사실만으로 더 만족스러운 추천이라고 할 수 있는가?

더 공부할 자료

- 임일, 『AI 에이전트를 위한 개인화 추천 알고리즘: Python, 머신러닝, AI, LLM 활용』, 청람, 2025, 2.4와 2.5(pp.27–32의 내용 기반 필터링). 교재의 용어와 내용 유사도 흐름을 사용했다. 최소제곱 해석, 작은 표, 장르 프로필과 네 방법의 공통 평가는 수업용 추가 설계다.
- 수업 재사용 API와 정의 링크: 인자·반환값·학습 속성·오류 및 원본 구현을 함께 확인한다.
- GroupLens MovieLens 100K: 데이터 소개와 사용 조건. 원본이나 교재 줄거리 데이터를 강의 ZIP에 포함하지 않는다.
- 데이터 누수, 내용 기반 추천, TF-IDF API, NDCG API: 본문의 개념과 실습을 보충하는 공식 자료(2026-09-09 확인).

코드 정의에서 보충 학습하기

API: arguments, results and examples

- build_comparison_app: 학습된 model을 사용하는 gradio.Blocks 객체를 만들고 반환한다.
- prepare_movielens: 캐시/다운로드/명시한 사본을 준비하고 데이터 종류까지 반환한다.
- split_ratings: ratings의 행 위치를 한 번 나누어 RatingSplit(train,test,method)를 반환한다.
- evaluate_means: 동일한 split에서 세 평균 회귀모형을 학습·평가해 DataFrame을 반환한다.
- FourMethodRecommender: 평점 수·영화 평균·집단 평균·장르 프로필을 같은 후보에서 비교한다.
- evaluate_rankings: 학습 완료 model을 고정하고 동일 test/후보/사용자에서 네 방법을 평가한다.
- MeanRatingPredictor: 관측 평점을 이용해 범주별 상수 회귀모형을 학습하는 클래스.
- MeanRatingPredictor.fit: ratings의 정답 평점으로 평균 모수를 추정하고 self를 반환한다.
- MeanRatingPredictor.predict: pairs(user_id/movie_id 표)의 평점 예측 Series를 반환한다.